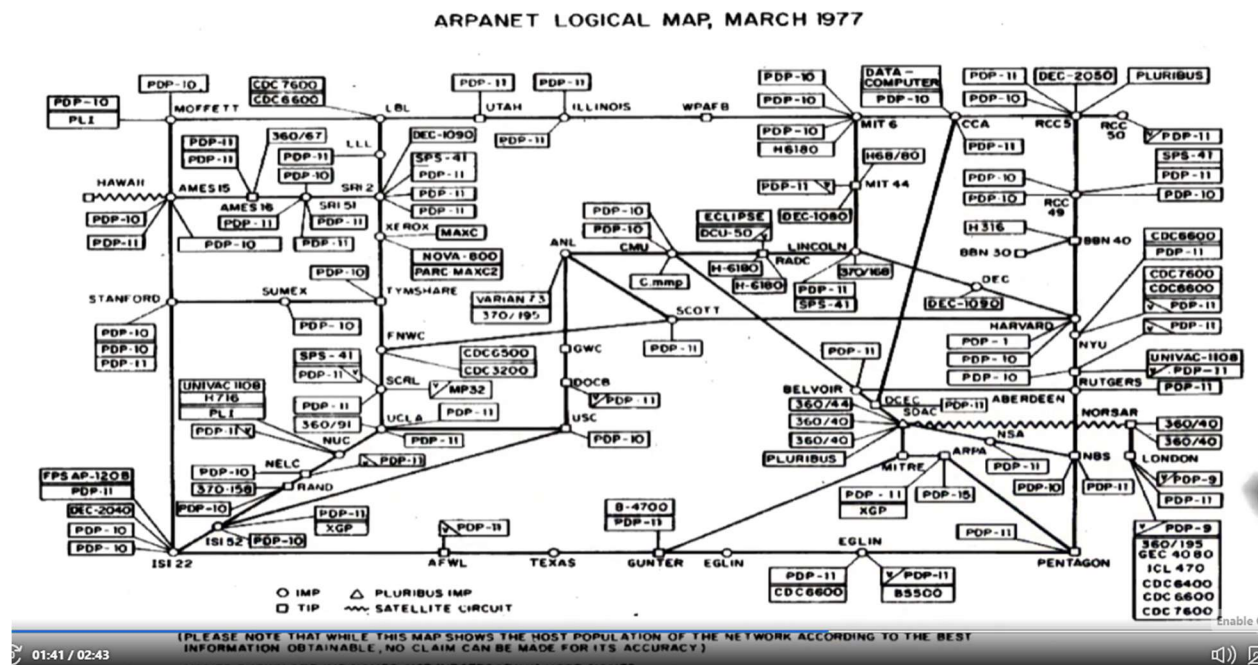


All the Computers in the World in one chart!!



The entire "Internet" network diagrammed here on one page! (1977)

Terminologies

When to release info?

- privacy vs. security
- Protection
authentication

Comprehensive set of security concerns

- unauthorized information release
- unauthorized information modification
- unauthorized denial of use

⇒ First mention of DOS attacks

Goal of secure system

- Prevent all violations ⇒ negative statement
↓
hard to achieve

05:16 / 05:48

Classic CIA triad (confidentiality, integrity, access) that should be provided by a secure system for data held on it.

Levels of Protection

Unprotected

e.g. MS-DOS... (hooks for **mistake prevention**
not same as security)

All or nothing

e.g. VM-370 + most time-sharing systems

Controlled sharing

e.g. access lists for files

User programmed sharing controls

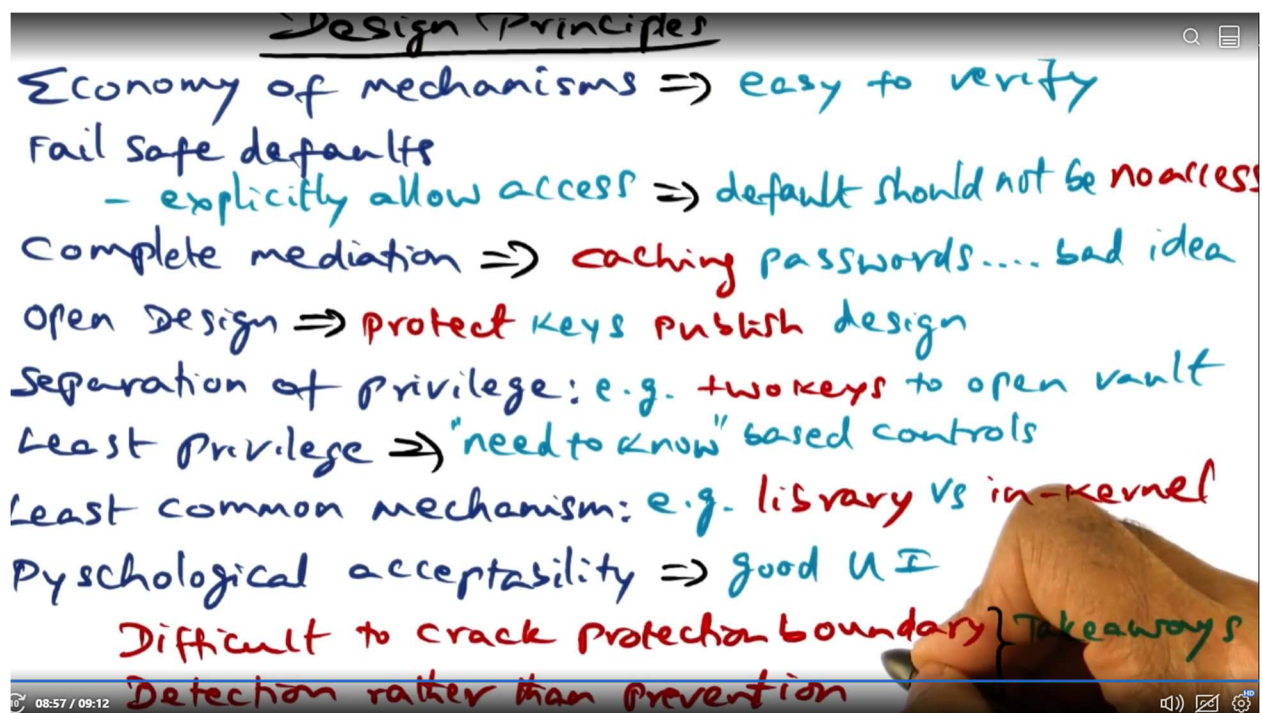
e.g. Unix like semantics for files

Strings on into

e.g. **"Top Secret"**

Need to deal with dynamics of use

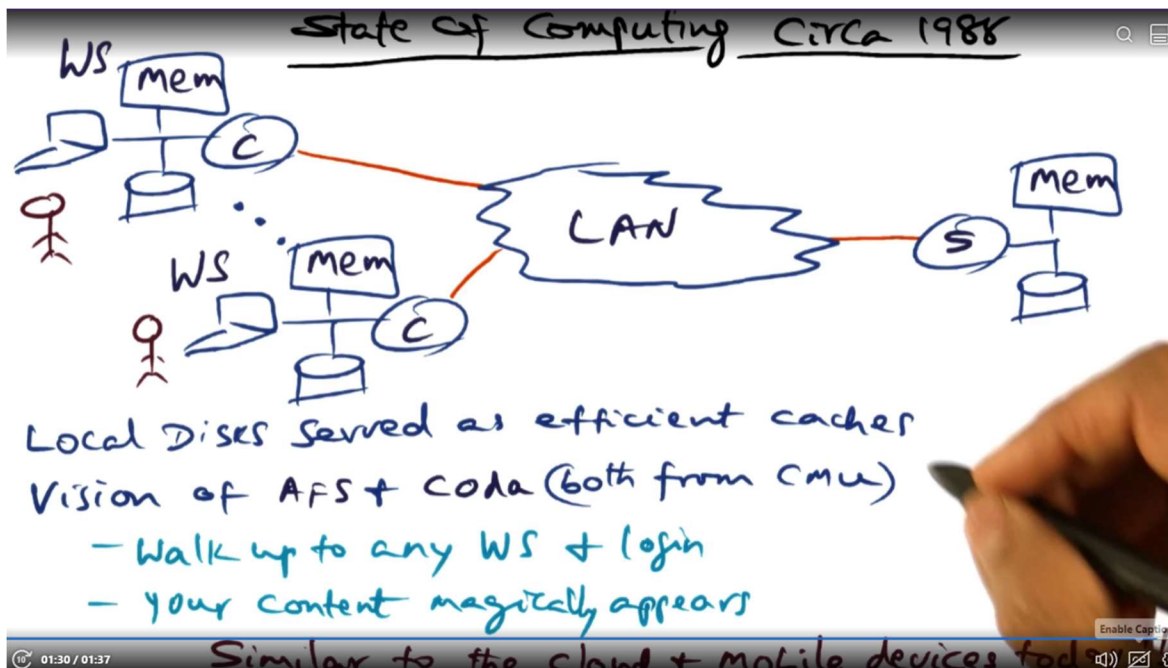
Different levels of protection, developed over time. The user-programmed sharing controls are still common today and used with Unix systems.



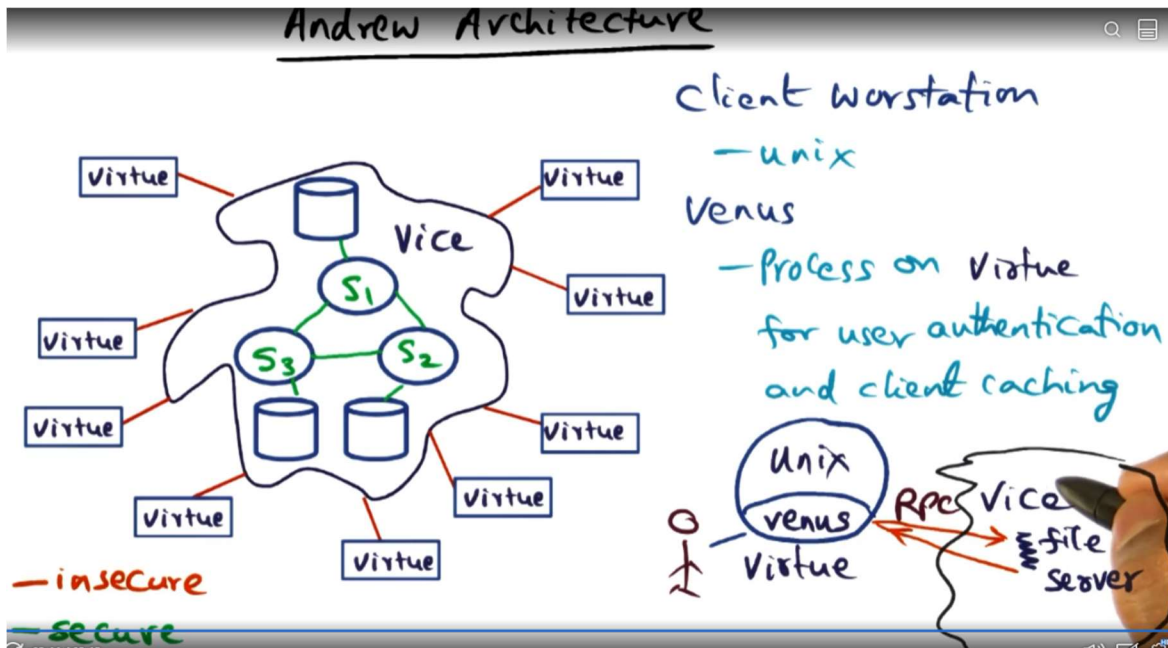
Positive requirements/stipulations – what a secure system “should” do instead of what it “should not” do, as opposed to a negative requirement like “prevent all violations.” Additionally, secure systems should focus on detection of violations instead of prevention of violations, because prevention is likely impossible and detection is much easier to confirm.

L11b

Andrew File System – experiment on CMU’s campus in 1988 that provided access to files from a centralized server to any workstation on campus by a LAN connection and using private key cryptography to send files over an untrusted/insecure network. Files can be cached on local workstations for quicker access.



AFS was integrated on CMU's campus in 1988 to serve files for users from any workstation on campus.



The workstations are considered "virtue" nodes, with unsecured links to the central "vice" network that has servers with data and is a locally secured network. The Venus process on the virtue workstations perform the necessary functions for authentication with the vice network and the file transfer and caching services.

Encryption Primer

Private Key System

— Symmetric Keys (e.g. passwords used in login)

S: Data $\rightarrow$ Enc (Data, key) $\rightarrow$ Cyphertext

R: Data $\leftarrow$ Dec (Cyphertext, key) $\leftarrow$

Public Key system

— Asymmetric Keys (Pair of Keys)

* Public Key Published $\Rightarrow$ encrypt } one way functions
* Private Key $\Rightarrow$ decrypt

S: Data $\rightarrow$ Enc (Data, public key) $\rightarrow$ Cyphertext

R: Data $\leftarrow$ Dec (Cyphertext, private key) $\leftarrow$

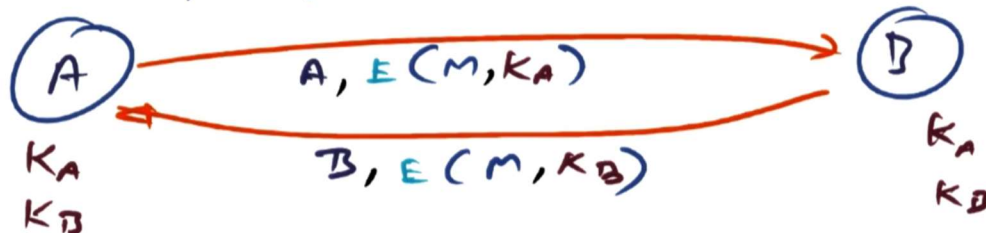
The private key system uses symmetric keys (same key for encryption/decryption) that are not shared with anyone except the sender/receiver of a message. The private key system with symmetric keys enables fast and efficient encryption/decryption of messages, but does not scale particularly well because each pair of interlocutors needs a unique private key shared between them, and sharing the key initially without revealing it to a third party can be difficult. This system is used with AFS because there are limited number of users and they can be assigned individual private keys to correspond with the shared file servers.

Public key systems use asymmetric keys (a public and private key) such that one key can be used to encrypt a message and the other can be used to decrypt a message. For example, a public key can be used to encrypt a message designated for the public key holder, who is then the only entity that can decrypt the message using their private key, which is never shared with anyone else. This system scales better (one public/private key pair for each entity in the system) and solves the problem of the initial transfer/sharing of private keys (because private keys are never shared, and public keys are visible to all). However, the encryption and decryption procedures are slower and more computationally expensive.

Private Key Encryption System in Action

With private key encryption

A + B have exchanged keys



Identity of sender

cleartext for receiver to know

what key to use for decryption

(K_A + K_B can be the same)

Private key encryption is used in this case (AFS) because of the limited number of users, and clear need for secure communication over an insecure LAN network.

Challenges for Andrew System

Authenticate User

Authenticate Server

Prevent replay attacks

isolate users

Choice in Andrew: Private key crypto system

⇒ identity of sender in cleartext

Traditional Unix: Username, Password

— overuse of L0K Security hole

Dilemma: What to use as identity and Private Key?

The user must authenticate with the server, and vice versa, to ensure files are transferred to the correct recipient and messages reach their intended target (not “spoofed” or impersonated).

Replay attacks involve a third party resending messages that have already been sent to gain more information from the server or client. Users should have isolated domains of files they can access and modify, without being able to access or modify other users’ files.

Andrew Solution

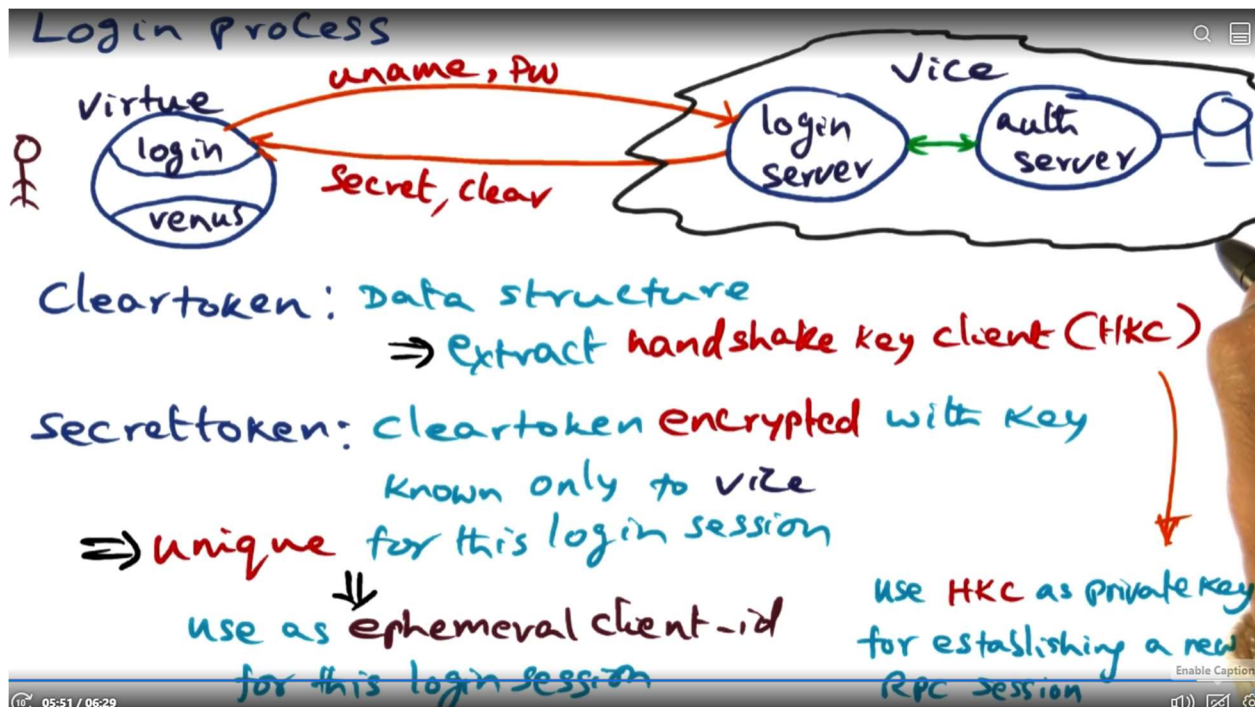
- username and password only for login
- use ephemeral id and keys for subsequent Venus-Vice communication

Three classes of client-server interaction

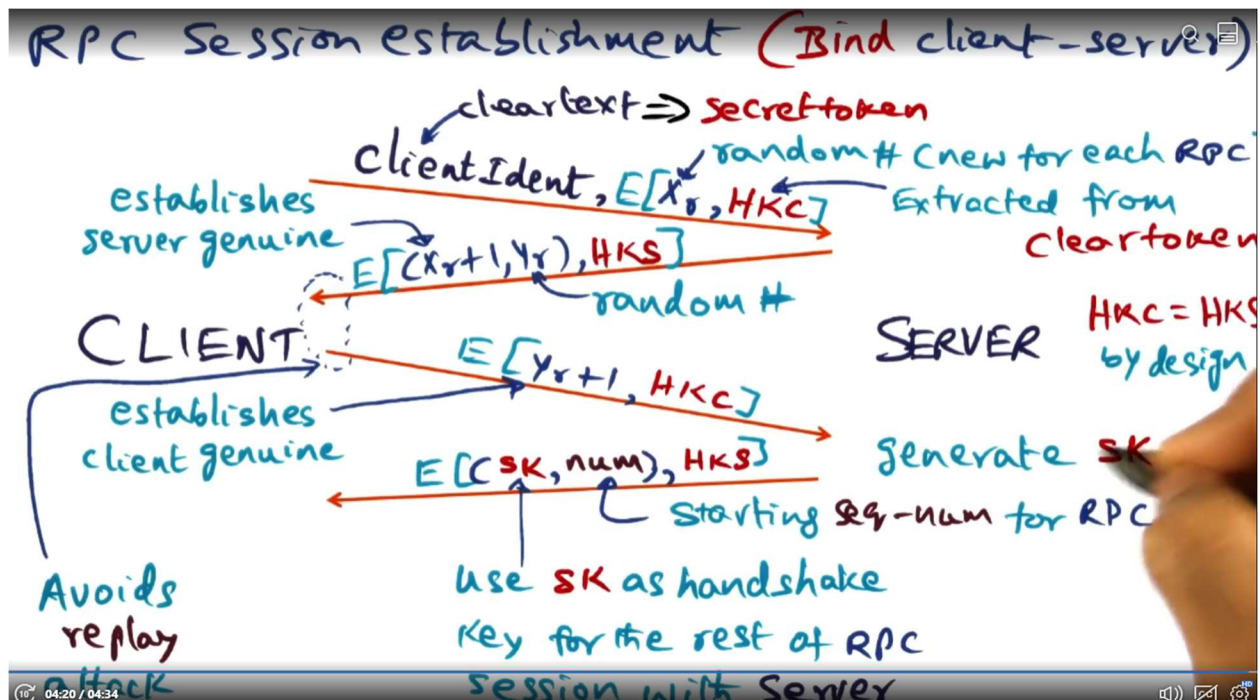
- login $\Rightarrow$ username, password
- RPC session establishment
- file system access during session

Both use ephemeral id and keys

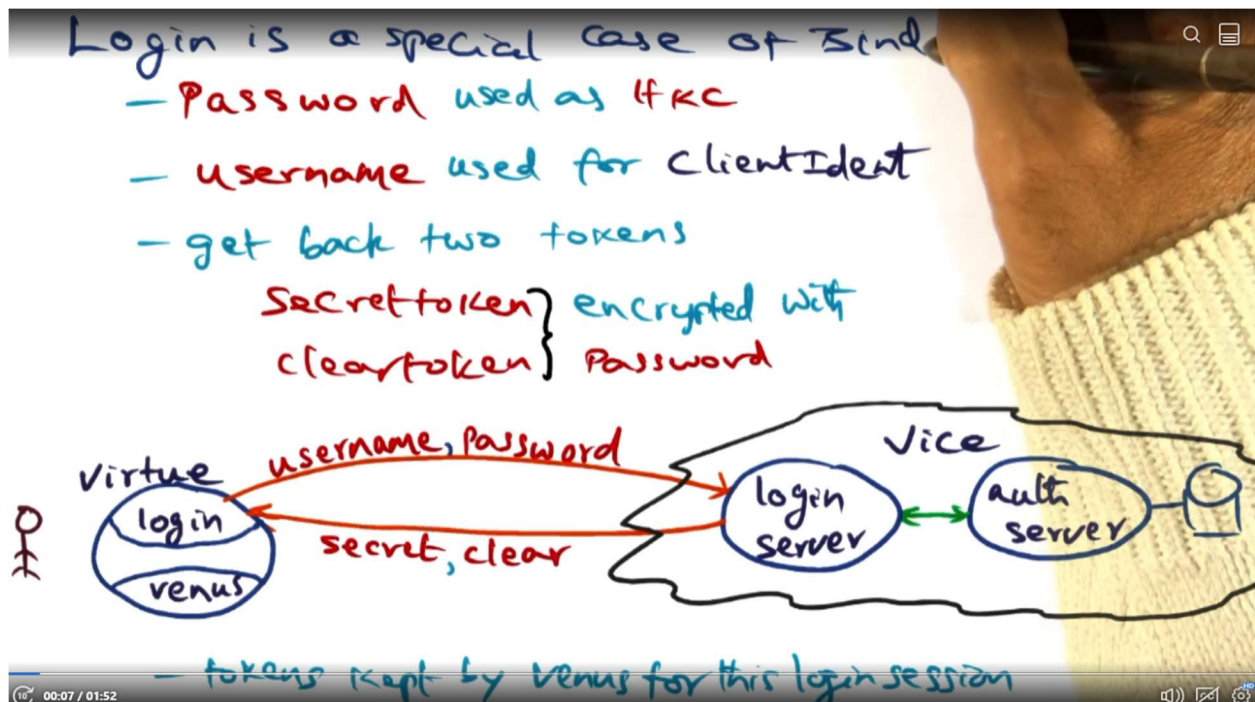
For AFS, symmetric cryptography was used to encrypt messages over the unsecured network to the "vice" server group that could be read by third parties. This cryptographic solution required a login event (registration with the server, authenticating the client and verifying the server for the client as well – two-way authentication), with information transferred during this event that would be used for later communication (ephemeral keys per login session, to setup later RPC sessions and file system operations).



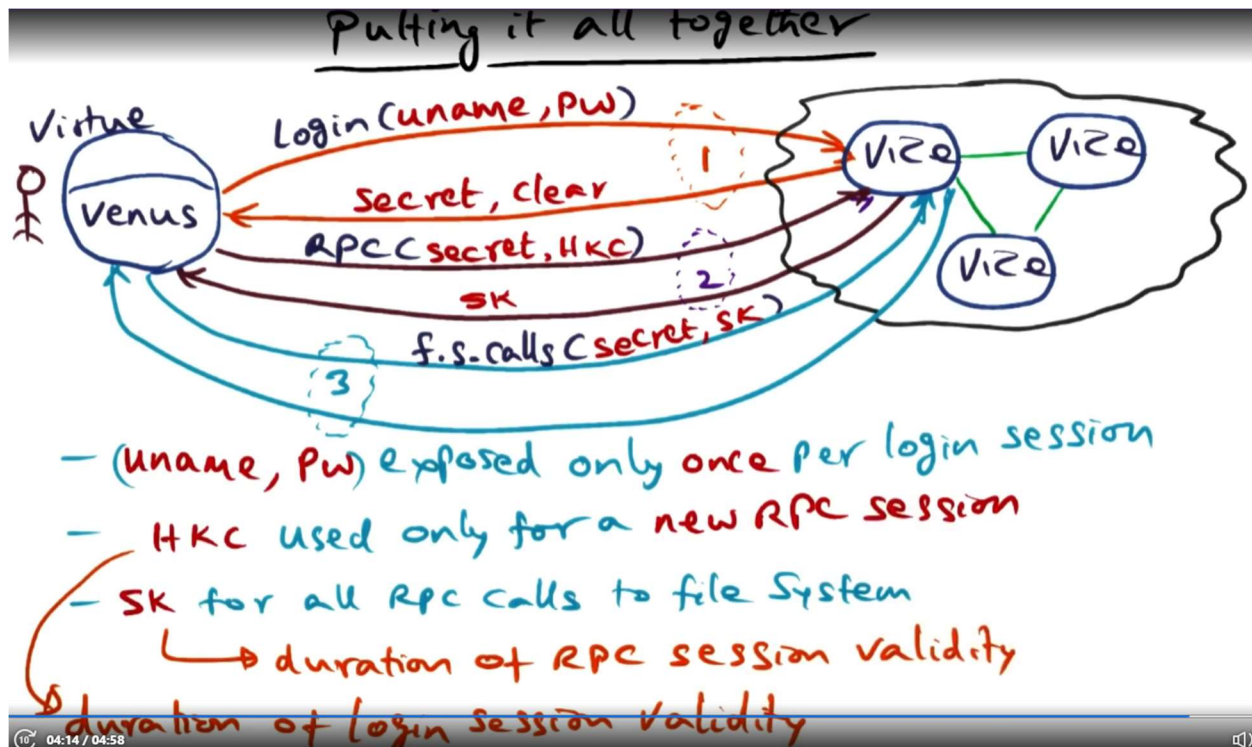
A user's username and password could be exposed and raise security concerns over the open network with multiple sends/receives, so other identification is used for validating later messages and continuing communication between the client and server. This other identification (ephemeral id and keys) is setup during the login for the session.



The RPC session is established after login, and uses the handshake key (HKC) to encrypt the client identity in the clear text, which can then be decrypted by the server to verify the same secret token was used with the handshake key, authenticating the client to the server. Additionally, a random number (X) is encrypted and sent to the server in this message from the client, which will then be incremented by one and re-encrypted using the same HKC (HKS = HKC), to authenticate the server with the client and prevent replay attacks. Additionally, another random number (Yr) will be sent to the client, and the client sends that number, incremented by one and then encrypted, back to the server as a final authentication step for the client while also preventing replay attacks. Then the server can generate a secret key (SK) specific to this RPC session, which will be used with all RPC calls for the duration of the session for this client and its file operation calls. This SK and a number to start and increment on each RPC call to prevent replay attacks are encrypted and sent from the server to the client, to be used as the handshake key (not the former HKC/HKS) to provide encryption for the following RPC calls.



The login event works much like the previously described RPC session establishment, but use the username and password for the clientIdent and HKC variables. The secret token and clear token are received at the client back from the server after the initial login message, encrypted by the password, which the client can then decrypt to obtain the secret and clear tokens (with the clear token containing the HKC). These tokens are stored in the venus process for the duration of this login session.



This image shows the possible messages/events that can be handled by the virtue client and vice server group for the Andrew File System at CMU. There can be multiple RPC sessions per login session, and multiple RPC calls per RPC session, but keys are only created/exchanged once for each session at the beginning to setup the later calls. This prevents replay attacks and helps ensure security for messages over the open network.